

WEEK 6

Aim:

To explore and implement the Node.js built-in os, path, dns, and net modules for performing system information retrieval, file path handling, DNS lookup, and TCP communication.

Description:

This experiment demonstrates the use of four built-in Node.js modules. The os module is used to obtain operating system, CPU, and memory information. The path module is used to process file paths. The dns module is used to find the IP address of a domain name. The net module is used to create a simple TCP server and communicate with a client.

Programs included:

1. OS Module – Display operating system platform, CPU architecture, CPU information, total memory, and free memory.
2. Path Module – Accept a file path and display the directory name, file name, extension, and normalized path.
3. DNS Module – Accept a domain name and display its corresponding IP address using DNS lookup.
4. Net Module – Create a TCP server, display a message when a client connects, and send a welcome message to the client.

Program Code:

1. Os.js

```
const os = require("os");
console.log("Platform:", os.platform());
console.log("Architecture:", os.arch());
console.log("CPU Information:", os.cpus());
console.log("Total Memory:", os.totalmem());
console.log("Free Memory:", os.freemem());
```

```
C:\Windows\System32\cmd.e x + v
Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.

C:\Users\laksh\OneDrive\Desktop\fullstack\week6>node os.js
Platform: win32
Architecture: x64
CPU Information: [
  {
    model: '11th Gen Intel(R) Core(TM) i5-11320H @ 3.20GHz',
    speed: 3187,
    times: {
      user: 2134593,
      nice: 0,
      sys: 2885296,
      idle: 19956484,
      irq: 417062
    }
  },
  {
    model: '11th Gen Intel(R) Core(TM) i5-11320H @ 3.20GHz',
    speed: 3187,
    times: {
      user: 1603562,
      nice: 0,
      sys: 1657687,
      idle: 21715078,
      irq: 87687
    }
  },
  {
    model: '11th Gen Intel(R) Core(TM) i5-11320H @ 3.20GHz',
    speed: 3187,
    times: {
      user: 2353890,
      nice: 0,
      sys: 2201562,
      idle: 20420875,
      irq: 114328
    }
  }
],
{
}
```

```

}
}
{
  model: '11th Gen Intel(R) Core(TM) i5-11320H @ 3.20GHz',
  speed: 3187,
  times: {
    user: 1354718,
    nice: 0,
    sys: 1530921,
    idle: 22090593,
    irq: 76671
  }
},
{
  model: '11th Gen Intel(R) Core(TM) i5-11320H @ 3.20GHz',
  speed: 3187,
  times: {
    user: 1770140,
    nice: 0,
    sys: 1859328,
    idle: 21346843,
    irq: 83625
  }
},
{
  model: '11th Gen Intel(R) Core(TM) i5-11320H @ 3.20GHz',
  speed: 3187,
  times: {
    user: 1412171,
    nice: 0,
    sys: 1486593,
    idle: 22077546,
    irq: 64843
  }
}
]
Total Memory: 8362713088
Free Memory: 925630464
C:\Users\laksh\OneDrive\Desktop\fullstack\week6>
```

2. Path.js

```
const path = require("path");

let filePath = "C:\\Users\\Lasya\\Documents\\test.txt";

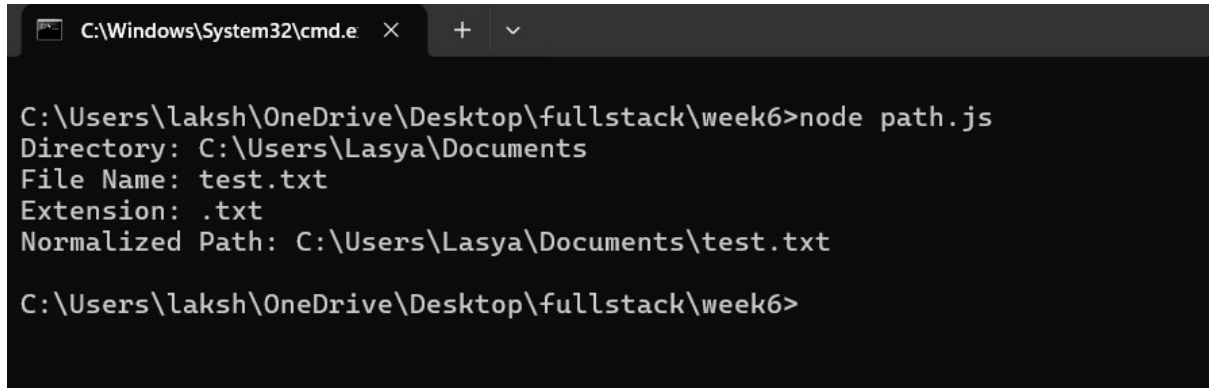
console.log("Directory:", path.dirname(filePath));

console.log("File Name:", path.basename(filePath));

console.log("Extension:", path.extname(filePath));
```

```
console.log("Normalized Path:", path.normalize(filePath));
```

Output:

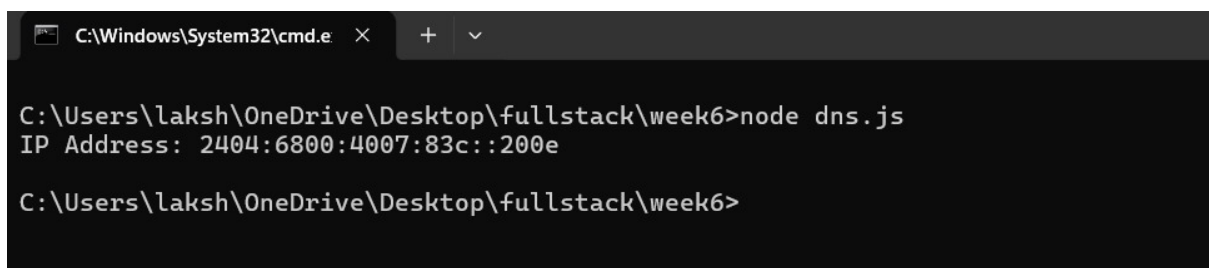
A screenshot of a Windows command prompt window. The title bar shows 'C:\Windows\System32\cmd.e' with a close button. The terminal content shows the execution of 'node path.js' from the directory 'C:\Users\laksh\OneDrive\Desktop\fullstack\week6'. The output displays the file name 'test.txt', its extension '.txt', and the normalized path 'C:\Users\Lasya\Documents\test.txt'.

```
C:\Windows\System32\cmd.e  X  +  v  
C:\Users\laksh\OneDrive\Desktop\fullstack\week6>node path.js  
Directory: C:\Users\Lasya\Documents  
File Name: test.txt  
Extension: .txt  
Normalized Path: C:\Users\Lasya\Documents\test.txt  
C:\Users\laksh\OneDrive\Desktop\fullstack\week6>
```

3) dns.js

```
const dns = require("dns");  
let domain = "google.com";  
dns.lookup(domain, (err, address) => {  
  if (err)  
    console.log("Error:", err.message);  
  else  
    console.log("IP Address:", address);  
});
```

Output:

A screenshot of a Windows command prompt window. The title bar shows 'C:\Windows\System32\cmd.e' with a close button. The terminal content shows the execution of 'node dns.js' from the directory 'C:\Users\laksh\OneDrive\Desktop\fullstack\week6'. The output displays the IP address '2404:6800:4007:83c::200e'.

```
C:\Windows\System32\cmd.e  X  +  v  
C:\Users\laksh\OneDrive\Desktop\fullstack\week6>node dns.js  
IP Address: 2404:6800:4007:83c::200e  
C:\Users\laksh\OneDrive\Desktop\fullstack\week6>
```

4) net module

4.1 server.js

```
const net= require("net");

const server=net.createServer((socket)=>{
  console.log("Client connected ");
  socket.write("Welcome to the TCP Server!");
  socket.on("end",()=>{
    console.log("Client disconnected");
  });
});

server.listen(3000,()=>{
  console.log("Server is running on port 3000");
});
```

4.2 client.js

```
const net=require("net");
const client= net.createConnection( {port:3000},()=>{
  console.log("Connected to server");
});
client.on("data",(data)=>{
  console.log("Server : ",data.toString());
  client.end();
});
```

Output:

```
Command Prompt
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\laksh>cd week5b

C:\Users\laksh\week5b>node client.js
Connected to server
Server : Welcome to the TCP Server!

C:\Users\laksh\week5b>
```

```
Command Prompt - node ser
C:\Users\laksh\week5b>node server.js
Server is running on port 3000
Client connected
Client disconnected
|
```

Details About Program:

Saved in: Week 6 folder of the repository.

Technology used: Node.js.

Execution: The programs are executed using the Node.js runtime through CMD/Terminal.

OS: The os module provides system and memory information.

Path: The path module processes and extracts information from a file path.

DNS: The dns module performs a DNS lookup and obtains the IP address of a domain.

Net: The net module creates a TCP server. The server listens for a client connection and sends a welcome message to the client.

Result:

The Node.js os, path, dns, and net modules were successfully implemented to perform system information retrieval, file path processing, DNS lookup, and TCP client-server communication.